time interaction – so your job is to create useful dumps and inspect them.

Remember: Always dump waveforms early. Even when you think it's obvious.

The 7 beginner bugs that waste the most time

Bug #1: Confusing wires and regs (in classic Verilog)

- `wire` is driven by continuous assignments or module outputs
- `reg` is a variable you assign inside procedural blocks (`initial`, `always`)

Modern SystemVerilog lets you use `logic`, which reduces confusion, but starting with classic Verilog forces you to understand drivers.

Bug #2: Accidental latches

You write combinational logic in an `always @(*)` block but forget an `else` or default assignment, so the signal "remembers" old values. That creates a latch.

Rule: In combinational always blocks:

- give defaults first, then override based on conditions

Bug #3: Blocking vs non-blocking assignment confusion

- Use = (blocking) for combinational procedural logic
- Use <= (non-blocking) for sequential logic on clock edges

Bug #4: Reset done wrong

Reset issues create "works sometimes" nightmares. Keep resets consistent and make sure your testbench applies them cleanly relative to clock edges.

Bug #5: Uninitialized signals in testbench

If you never assign a reg, it starts as X. Your whole design becomes "X soup."

Bug #6: Time units mismatch

Your testbench uses `#10` thinking it's 10ns but your `timescale` makes it something else. Set timescale clearly.

Bug #7: Not checking outputs

If your testbench just "runs," you can miss silent wrong behavior. Add checks.

Mini-build 1: Counter + clock divider

RTL: counter.v

```
module counter #(
    parameter WIDTH = 8
)(
    input              clk,
    input              rst_n,
    input              en,
    output reg [WIDTH-1:0] q
);
    always @(posedge clk) begin
        if (!rst_n)
            q <= {WIDTH{1'b0}};
        else if (en)
            q <= q + 1'b1;
    end
endmodule
```

Testbench: tb_counter.v

```
`timescale 1ns/1ps

module tb_counter;
    reg clk, rst_n, en;
    wire [7:0] q;

     counter #(.WIDTH(8)) dut (.clk(clk),
.rst_n(rst_n), .en(en), .q(q));

    // Clock: 10ns period
    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("waves/counter.vcd");
        $dumpvars(0, tb_counter);

        rst_n = 0; en = 0;
        #20;
        rst_n = 1; en = 1;

        #200;
        en = 0;
        #50;
        en = 1;
        #50;

        $finish;
    end
endmodule
```

Step-by-step run

```
iverilog -o build/tb_counter tb/tb_counter.v
src/counter.v
vvp build/tb_counter
gtkwave waves/counter.vcd
```